



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Memo No. _____

Date / /

(91) Equivalence of CFG and PDA

Theorem: A Language is Context Free iff some Pushdown Automata recognises it

Proof:

part 1: $CFG \rightarrow PDA$

part 2: $PDA \rightarrow CFG$

Given a grammar $S \rightarrow BS \mid A$

$A \rightarrow 0A \mid \epsilon$

$B \rightarrow BB \mid 1$

Find or

build a PDA

(left most derivation)

$\rightarrow S$

$\rightarrow BS$

$\rightarrow BB \mid S$

$\rightarrow 22 \mid S$

$\rightarrow 22 \mid A$

$\rightarrow 22 \mid \epsilon$

$\rightarrow 22 \mid \epsilon \rightarrow 22$

General form:

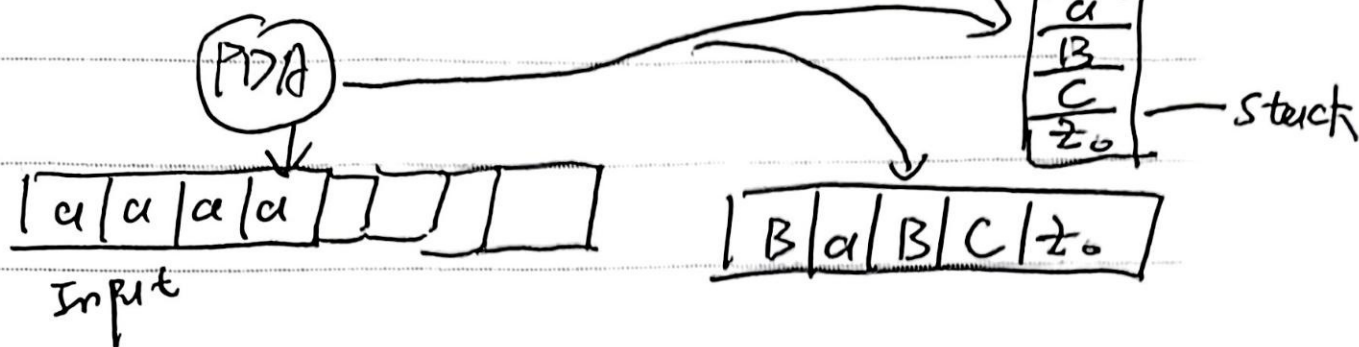
$\underbrace{aaaa}_{\text{Terminals}} \underbrace{BaBC}_{\text{Rest}}$



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Memo No. _____

Date _____



Left Most Derivation:

$S \rightarrow \dots aaaa \boxed{B/a/B/C} \rightarrow \dots$

AT EACH STEP EXPAND LEFT-MOST DERIVATION

Eg. Rule: $B \rightarrow ASAxB$

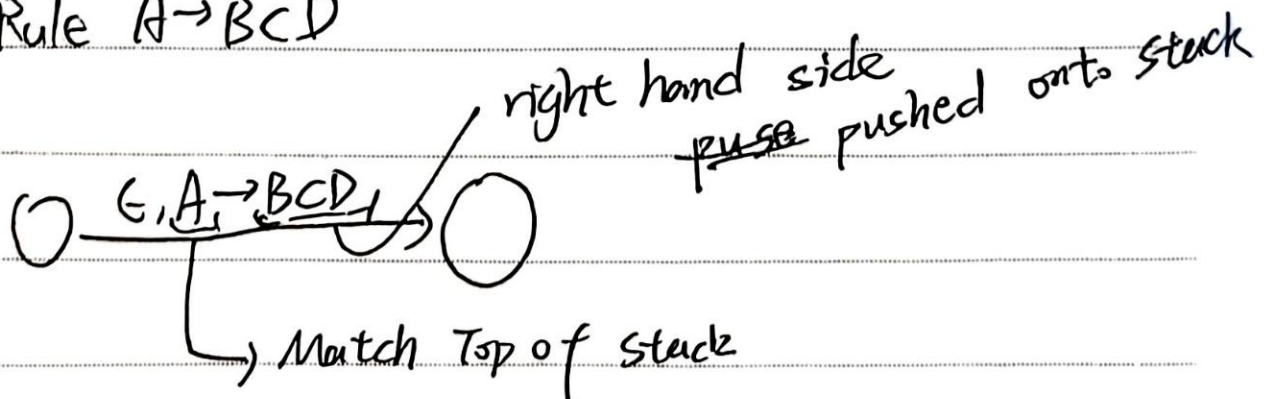
$\rightarrow \dots aaaa \boxed{A/S/A/x/B/a/B/C}$

$\rightarrow$ Match Stack Top to a Rule

$\rightarrow$ Pop Stack

$\rightarrow$ Push Right Hand Side of Rule onto Stack

Ex: Rule $A \rightarrow BCD$

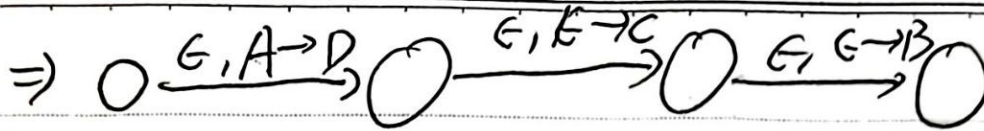




Mo Tu We Th Fr Sa Su

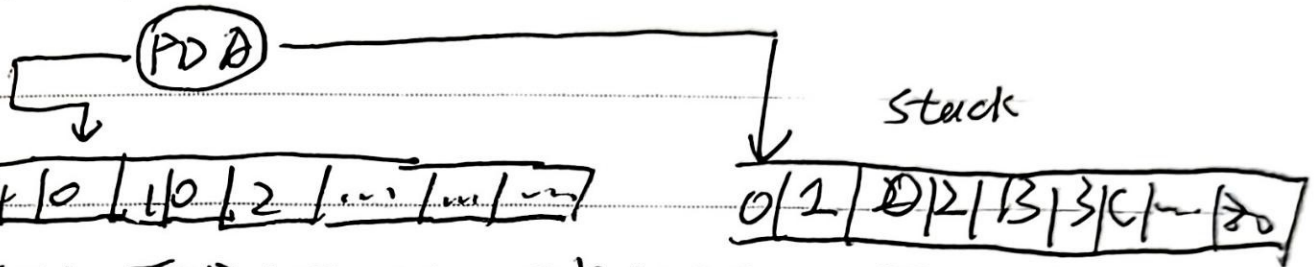
Memo No. _____

Date / /

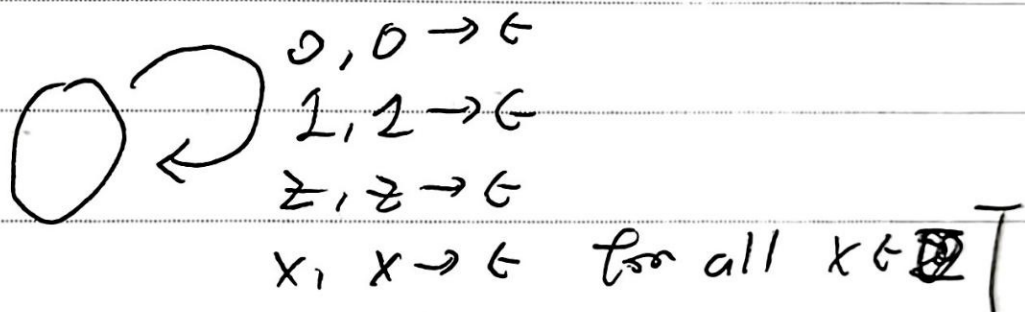


If have Terminal symbol

Rule : $A \rightarrow 0102B3C$

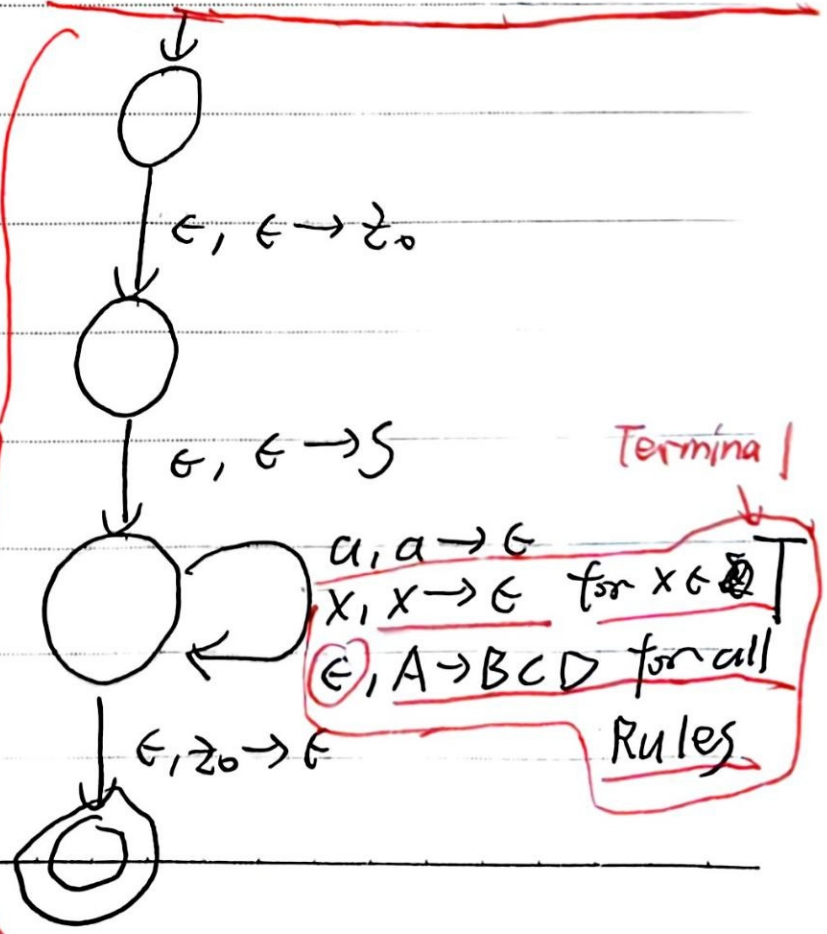


MATCH TERMINAL SYMBOLS TO THE STACK TOP



The Final PDA

can represent
all CFGs
just expand
 $\epsilon, A \rightarrow BCD$





Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Memo No. _____

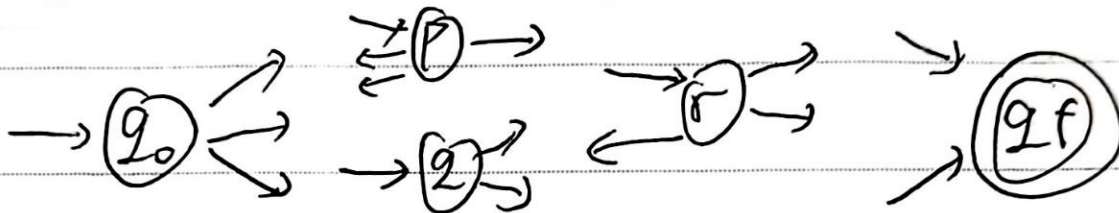
Date / /

Q2) Part 2, PDA $\rightarrow$ CFG

Given a PDA $\rightarrow$ Build a CFG from it

Step 1: Simplify the PDA

Step 2: Build the CFG



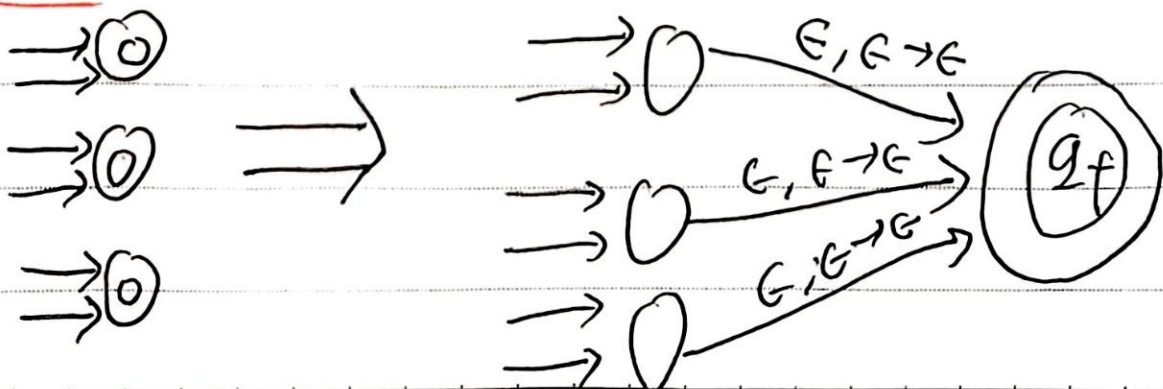
PDA

There will be a Non-Teriminal for every pair of states: A_{pq}, A_{qr}, A_{rq}

The starting Non Terminal will be: $A_{q_0 q_f}$

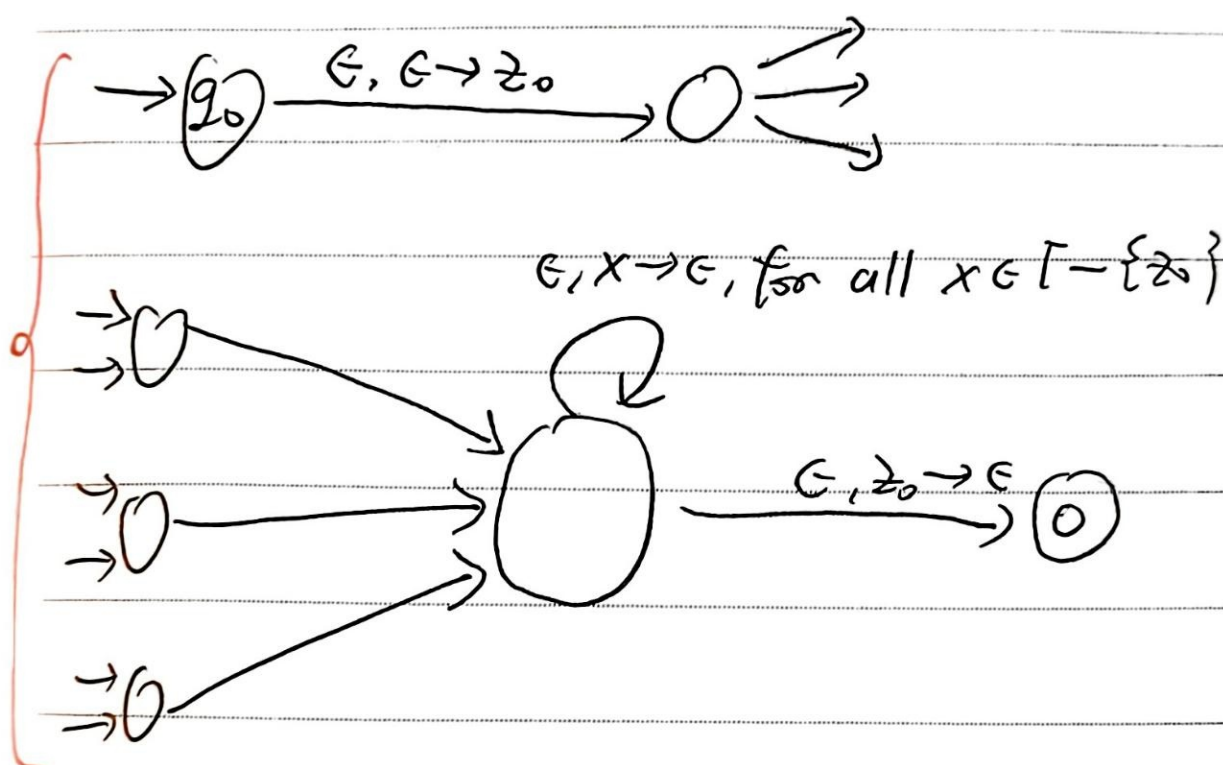
Step 1: Simplify the PDA:

1) The PDA should have only one final/accept state.

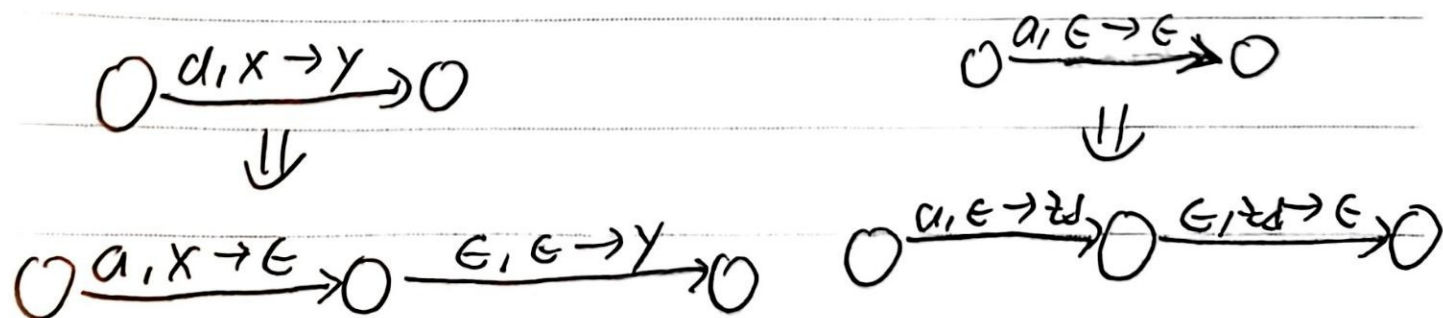


2) The PDA should empty its stack before accepting

→ Create a new State, Start State q_0 which pushes z_0 to the stack,



3) Make sure each transitions either Pushes or Pops but does not do both





Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Memo No. _____

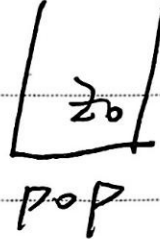
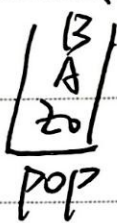
Date / /

~~3)~~

--> Start with an empty stack and
finish with an empty stack

~~UNDERFLOW~~

UnderFlow

not
allow:

OVERFLOW

(93) $PDA \rightarrow CFG$, step 2, Build the CFG
MAIN IDEA.

1. Consider q and p in the PDA.
states

2. Could we go from p to q without Stack Underflow and maintaining an empty stack at the beginning and ~~right~~^{end}? OR if something were already on the stack they would never be touched.

What strings would do there?

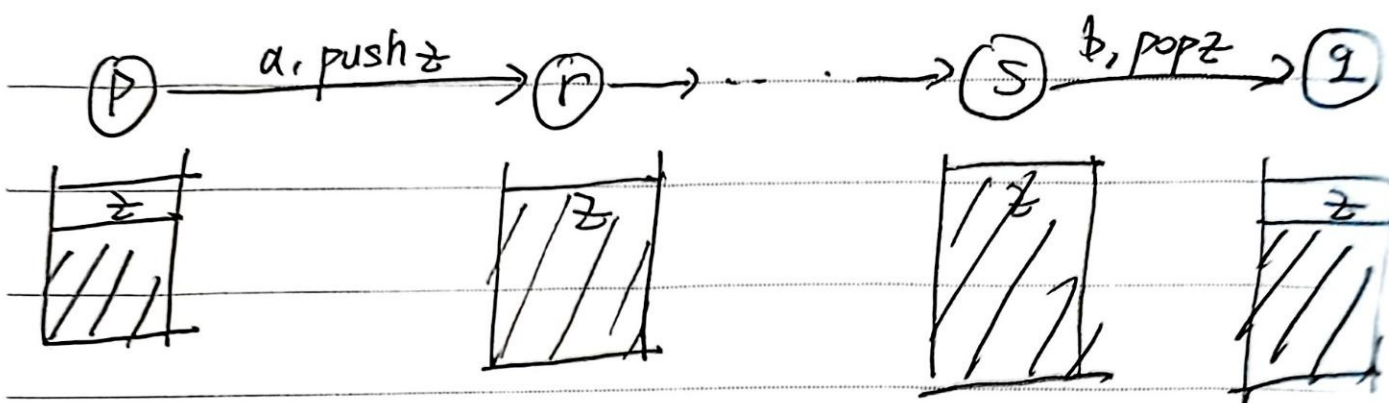
We will create a Non Terminal A_{pq} in our Grammar. A_{pq} will generate exactly those strings that will take us from p to q maintaining all the above stack conditions

Case-1.



Memo No. _____

Date / /



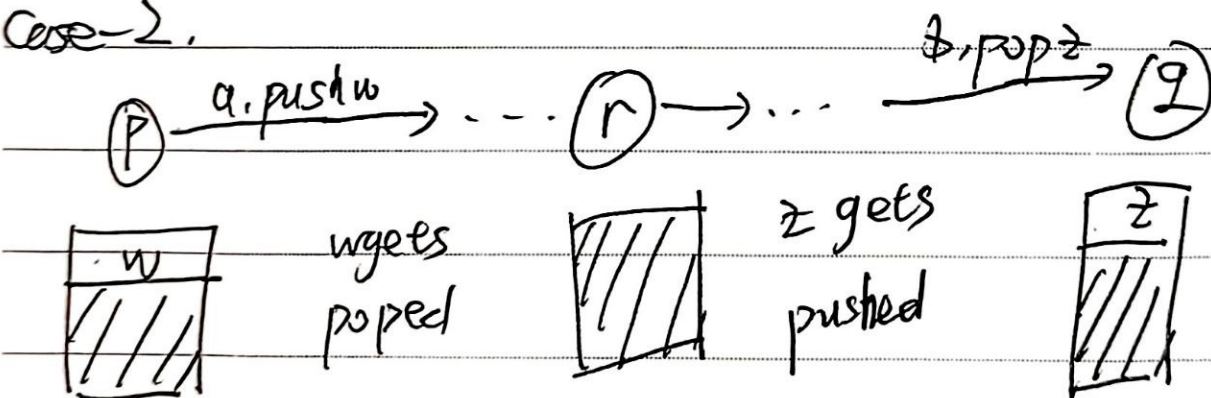
what strings can be generated by following this path?

$\rightarrow "a \dots b"$

$A_p q \rightarrow a A_r s b$

$\rightarrow$ This rule will generate exactly those strings.

Case-2.



what strings can be generated by following this path?

$A_p q \rightarrow A_p r A_q$

$\rightarrow \checkmark$



Memo No. _____

Date / /

- To get from any state (p) to itself. App $\rightarrow \epsilon$
- If the PDA accepts some strings, then there is a way to go from (q) to (q) that does not modify the stack

Our Start Non-Terminal is A_3, q_1